# CO2 ASSIGNMENT

C++ Programming – Classes, Objects and Number-Based Logic

| | | | |
|---|---|---|---|
| Name: | _____ | Register No.: | _____ |
| Department: | _____ | Subject: | _____ |

## QUESTION 1: Check Whether a Number Is a Sunny Number

**Aim:** A number is sunny if its successor (n + 1) is a perfect square.

**C++ Program:**

```
#include <iostream>
#include <cmath>
using namespace std;

class SunnyNumber {
private:
    long long n;

public:
    SunnyNumber(long long value) {
        n = value;
    }

    bool isSunny() {
        if (n < 0) return false;

        long long value = n + 1;
        long long root = (long long)sqrt((long double)value);

        return root * root == value;
    }
};

int main() {
    long long n;
    cin >> n;

    SunnyNumber obj(n);

    if (obj.isSunny())
        cout << n << " is a Sunny number";
    else
        cout << n << " is not a Sunny number";

    return 0;
}
```

| Sample Input | Expected Output |
|---|---|
| 8 | 8 is a Sunny number |

**Test Cases:**
8 → Sunny number
2 → Sunny number
10 → Not a Sunny number

**Explanation:** For 8, the successor is 9, which is $3^2$. For 2, the successor is 3—not a square; therefore the supplied test case 2 → Sunny number is mathematically inconsistent. The correct result for 2 is **not** a Sunny number. 10 + 1 = 11, which is not a square.

## QUESTION 2: Check Whether a Number Is a Fascinating Number

**Aim:** A three-digit number is fascinating when n, 2n and 3n, concatenated, contain each digit from 1 to 9 exactly once.

**C++ Program:**

```
#include <iostream>
#include <string>
using namespace std;

class NumberAnalyzer {
private:
    int n;

public:
    NumberAnalyzer(int value) {
        n = value;
    }

    bool isFascinating() {
        if (n < 100 || n > 999) return false;

        string s = to_string(n) +
                   to_string(2 * n) +
                   to_string(3 * n);

        if (s.length() != 9) return false;

        int count[10] = {0};

        for (char ch : s) {
            if (ch == '0') return false;
            count[ch - '0']++;
        }

        for (int d = 1; d <= 9; d++) {
            if (count[d] != 1) return false;
        }
        return true;
    }
};

int main() {
    int n;
    cin >> n;

    NumberAnalyzer obj(n);

    if (obj.isFascinating())
        cout << n << " is a Fascinating number";
    else
        cout << n << " is not a Fascinating number";

    return 0;
}
```

| Sample Input | Expected Output |
|---|---|
| 192 | 192 is a Fascinating number |

**Test Cases:**
192 → Fascinating number
219 → Fascinating number
123 → Not a Fascinating number

**Explanation:** For 192, the products are 192, 384 and 576. Their concatenation contains digits 1 through 9 exactly once. The same property holds for 219 (219, 438, 657).

## QUESTION 3: Check Whether a Number Is a Disarium Number

**Aim:** A number is Disarium if the sum of its digits raised to their positions from left to right equals the original number.

**C++ Program:**

```
#include <iostream>
#include <string>
#include <cmath>
using namespace std;

class DisariumNumber {
private:
    int n;

public:
    DisariumNumber(int value) {
        n = value;
    }

    bool isDisarium() {
        if (n <= 0) return false;

        string s = to_string(n);
        int sum = 0;

        for (int i = 0; i < (int)s.length(); i++) {
            int digit = s[i] - '0';
            sum += (int)pow(digit, i + 1);
        }

        return sum == n;
    }
};

int main() {
    int n;
    cin >> n;

    if (n <= 0) {
        cout << "Invalid input";
        return 0;
    }

    DisariumNumber obj(n);

    if (obj.isDisarium())
        cout << n << " is a Disarium number";
    else
        cout << n << " is not a Disarium number";

    return 0;
}
```

| Sample Input | Expected Output |
|---|---|
| 175 | 175 is a Disarium number |

**Test Cases:**
175 → Disarium number
123 → Not a Disarium number
0 → Invalid input

**Explanation:** For 175: $1^1 + 7^2 + 5^3 = 1 + 49 + 125 = 175$. Since the sum equals the original number, 175 is a Disarium number.

## QUESTION 4: Check Whether a Number Is a Kaprekar Number

**Aim:** A positive number is Kaprekar if its square can be split into two parts, with the right part having the same number of digits as the original number, and both parts sum to the original number.

**C++ Program:**

```cpp
#include <iostream>
using namespace std;

class KaprekarNumber {
private:
    long long n;

public:
    KaprekarNumber(long long value) {
        n = value;
    }

    bool isKaprekar() {
        if (n <= 0) return false;

        long long square = n * n;
        long long divisor = 1;
        long long temp = n;

        while (temp > 0) {
            divisor *= 10;
            temp /= 10;
        }

        long long right = square % divisor;
        long long left = square / divisor;

        return (left + right == n);
    }
};

int main() {
    long long n;
    cin >> n;

    if (n <= 0) {
        cout << "Invalid input";
        return 0;
    }

    KaprekarNumber obj(n);

    if (obj.isKaprekar())
        cout << n << " is a Kaprekar number";
    else
        cout << n << " is not a Kaprekar number";

    return 0;
}
```

| Sample Input | Expected Output |
|---|---|
| 45 | 45 is a Kaprekar number |

**Test Cases:**
45 → Kaprekar number
55 → Kaprekar number
10 → Not a Kaprekar number

**Explanation:** For 45, $45^2 = 2025$. Split it as 20 and 25. Their sum is 20 + 25 = 45, so 45 is a Kaprekar number.

## QUESTION 5: Check Whether a Number Is a Keith Number

**Aim:** A Keith number is formed by starting with its own digits; each next term is the sum of the previous terms, and the sequence must reach the original number.

**C++ Program:**

```
#include <iostream>
#include <vector>
using namespace std;

class KeithNumber {
private:
    int n;

public:
    KeithNumber(int value) {
        n = value;
    }

    bool isKeith() {
        if (n <= 0) return false;

        vector<int> digits;
        int temp = n;

        while (temp > 0) {
            digits.insert(digits.begin(), temp % 10);
            temp /= 10;
        }

        int count = digits.size();

        while (true) {
            int next = 0;

            for (int x : digits)
                next += x;

            if (next == n) return true;
            if (next > n) return false;

            digits.erase(digits.begin());
            digits.push_back(next);
        }
    }
};

int main() {
    int n;
    cin >> n;

    if (n <= 0) {
        cout << "Invalid input";
        return 0;
    }

    KeithNumber obj(n);

    if (obj.isKeith())
        cout << n << " is a Keith number";
    else
        cout << n << " is not a Keith number";

    return 0;
}
```

| Sample Input | Expected Output |
|---|---|
| 742 | 742 is a Keith number |

**Test Cases:**
742 → Keith number
14 → Keith number
123 → Not a Keith number

**Explanation:** For 197, the sequence begins 1, 9, 7, 17, 33, 57, 107, 197. Since it reaches 197, it is a Keith number. The same sequence rule is applied by the program to each input.

## QUESTION 6: Check Whether a Number Is a Trimorphic Number

**Aim:** A number is Trimorphic if its cube ends with the same digits as the original number.

**C++ Program:**

```
#include <iostream>
using namespace std;

class TrimorphicNumber {
private:
    long long n;

public:
    TrimorphicNumber(long long value) {
        n = value;
    }

    bool isTrimorphic() {
        if (n < 0) return false;

        long long cube = n * n * n;
        long long divisor = 1;
        long long temp = n;

        while (temp > 0) {
            divisor *= 10;
            temp /= 10;
        }

        return (cube % divisor == n);
    }
};

int main() {
    long long n;
    cin >> n;

    if (n < 0) {
        cout << "Invalid input";
        return 0;
    }

    TrimorphicNumber obj(n);

    if (obj.isTrimorphic())
        cout << n << " is a Trimorphic number";
    else
        cout << n << " is not a Trimorphic number";

    return 0;
}
```

| Sample Input | Expected Output |
|---|---|
| 24 | 24 is a Trimorphic number |

**Test Cases:**
24 → Trimorphic number
7 → Not a Trimorphic number
-5 → Invalid input

**Explanation:** For 24, $24^3 = 13824$. The last two digits are 24, which match the original number. Therefore, 24 is Trimorphic.

## Conclusion

The six C++ programs use classes, objects and member functions to check special number properties. Each program accepts an input, applies the required mathematical logic and displays the result. The sample test cases and outputs are included for reference.

**Note:** The test case '2 → Sunny number' in the supplied screenshot is incorrect under the given definition. Since 2 + 1 = 3 and 3 is not a perfect square, the correct result is '2 is not a Sunny number'.